

Our model's hybrid memory: KV cache & DeltaNet

How Qwen 3.6 "remembers" a conversation by combining two memories of opposite natures — and why that architectural choice drives our infrastructure settings.

Accessible read: the mechanisms are explained through analogies and diagrams. A final section ties everything back to our concrete deployment decisions.

1 How an AI "reads" and remembers

It all starts with one simple idea: words look at each other.

To understand a word, the model doesn't read it in isolation: it looks at the surrounding words to decide which ones matter. This mechanism — "attention" — is what gives a sentence its meaning.

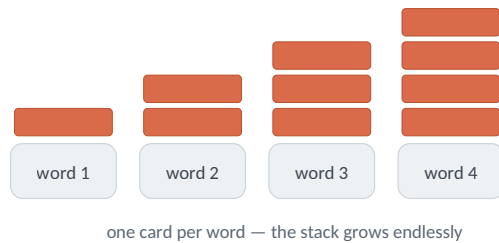
An example

"The mouse chewed through the cable." To resolve the ambiguity of "mouse" (the animal or the computer device?), the model relies on "chewed" and "cable." To follow an entire conversation, it must keep a trace of what it has read: that is its **memory**. And there are two very different kinds.

2 Two ways of remembering

The single most important distinction in this whole document.

KV cache — the ever-growing binder



DeltaNet — the single sheet

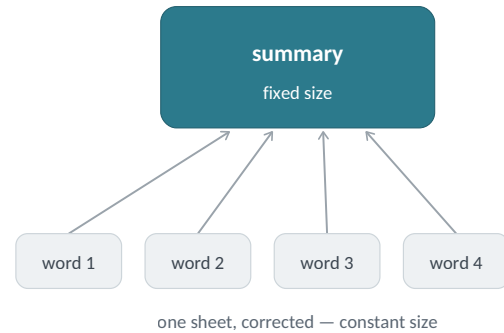


Diagram 1 — Two opposite memories: on the left, a trace kept per word (exact but heavy); on the right, a single fixed-size summary (light but approximate).

The binder (KV cache). The model keeps a trace for every word read and never discards anything. Perfect recall — it can retrieve a precise detail far back — but the stack grows with the conversation, and re-reading it gets more and more expensive.

The sheet (DeltaNet). The model keeps a single fixed-size summary, which it **corrects** at each word instead of stacking. Light and fast regardless of length — but the summary is a compression, so slightly approximate.

3 Our model combines both: "hybrid"

It doesn't use one memory *or* the other — it alternates between them, layer after layer.

The model is a stack of 40 layers that a word passes through entirely, bottom to top. Most layers (the DeltaNet layers) hold the **summary sheet**; a few (the attention layers) feed the **card binder**. The pattern "3 light layers + 1 precise layer" repeats 10 times. This mix is what we call a **hybrid** architecture.

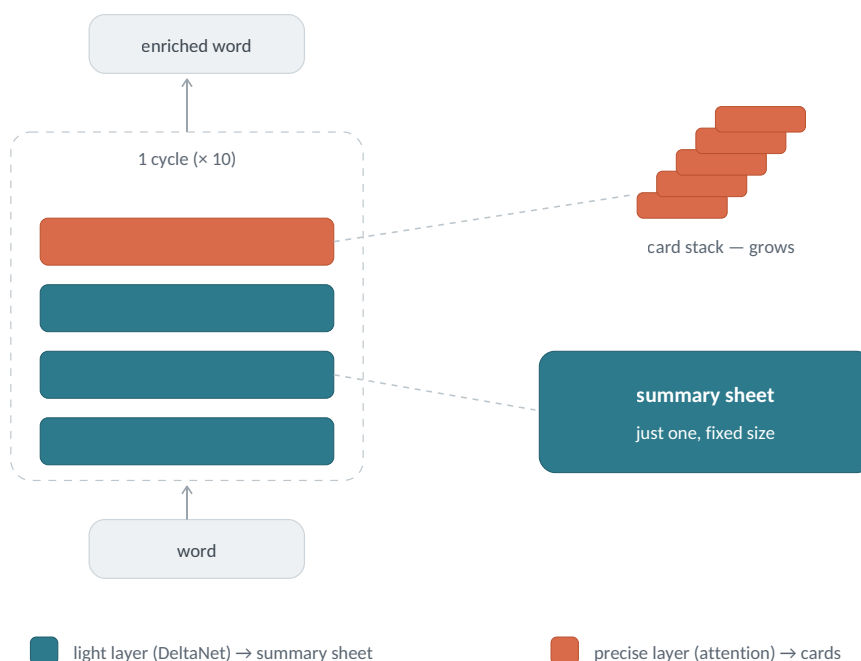


Diagram 2 — Coexistence: a word passes through all 40 layers. The light layers update the single summary sheet; the precise layers add cards to the stack. Both memories fill up in parallel.

The benefit of mixing: the model gets both the **lightness** of the DeltaNet layers (reading very long texts without slowing down) and the **precision** of the attention layers (retrieving an exact distant detail). A minority of "expensive" layers is enough to keep faithful recall; the rest stays economical.

4 How each one evolves over time

With each new word, the two memories behave in opposite ways.

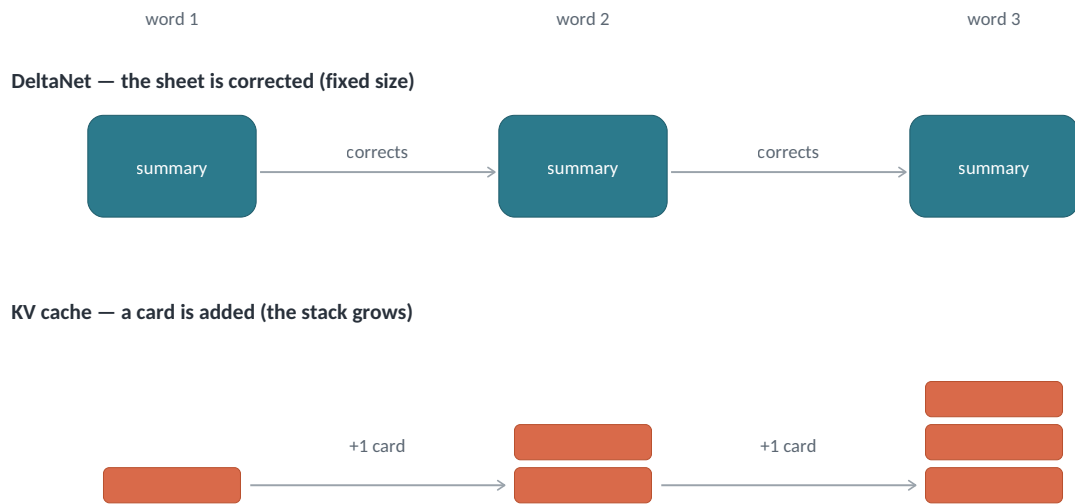


Diagram 3 — Dynamics: the summary sheet stays the same box (rewritten but never enlarged), while the stack gains one card per word.

In one sentence: the sheet is a **fixed-size compressed memory**; the binder is an **exact memory that grows**. Both run in parallel inside the hybrid model.

5 What this changes for our deployment

This architecture is not a theoretical curiosity: it directly explains our infrastructure choices.

Aspect	Binder (KV cache)	Sheet (DeltaNet)
Presence	~10 layers out of 40	~30 layers out of 40
Size vs. context	grows with the conversation	constant
Write precision	tolerant → fp8 (compact)	fragile → fp16 (precise)
Associated setting	<code>kv-cache-dtype fp8</code>	<code>mamba-cache-dtype float16</code>

Why long context is affordable for us

Only ~10 layers out of 40 carry a binder, and those also use header compression. As a result, at 128,000 words of context the binder weighs on the order of 1.3 GB, versus several times more for a conventional model of comparable size. That is what makes a very large context window viable on our single GPU.

Why two different precisions

The binder's cards are written once and never touched again: they can be compressed aggressively (fp8) with no risk. The summary sheet, however, is rewritten at every word; at reduced precision, tiny rounding errors would accumulate and the sheet would "drift." We therefore keep it at finer precision (fp16) — at a negligible memory cost, since it is fixed-size and tiny.

Why a dedicated prefill setting

Because the DeltaNet layers make up the majority of the model, the engine exposes a specific setting (`gdn-prefill-backend`) to optimize the phase where the model ingests the input text. This is one of the performance levers we are evaluating on our hardware through benchmarking.

6 Key takeaways

- The model combines **two memories**: an exact binder that grows, and a light fixed-size summary sheet.
- This **hybrid** mix delivers both cheap long context and precise recall.
- It makes a **very large context window affordable** on our single-GPU infrastructure.
- It justifies our settings: heavy compression of the binder (fp8), preserved precision for the sheet (fp16), and a dedicated engine for the DeltaNet part.

7 Mini-glossary

Attention	The fact that each word looks at the others to understand context.
KV cache	The "binder": a trace kept per word. Exact memory, but it grows.
DeltaNet	The single "summary sheet," corrected at each word. Light, fixed-size memory.
Hybrid	The alternation, within the same model, of DeltaNet layers and attention layers.
fp8 / fp16	Two levels of write precision: compact (fp8) or fine (fp16).
Prefill	The moment when the model ingests all the input text before answering.